

Exercise 2 – Data Structures and Algorithms

Developed by: Raian Vargas Maretto

Date: 18 November 2025

Overview

In this exercise, you will practice some data structures and algorithms with a relative amount of geospatial data. The exercise consists in loading a layer of points containing the coordinates of around 720 urban centers in Europe, extracted from the Global Human Settlement Layer (GHSL - <https://data.jrc.ec.europa.eu/dataset/53473144-b88c-44bc-b4a3-4583ed1f547e>), and perform a set of operations that will allow you to practice the concepts you saw in class. Despite you are not forbidden of using generative AI to do the exercises, you are strongly recommended to solve the exercises by yourself as much as possible. Remember that, the only way to become a good programmer is **putting your own hands in the code**. Try to practice by yourself as much as possible the concepts you've seen in the lecture.

The exercise consists in the following steps

1. Store spatial points (as city locations with coordinates).
2. Search for nearby points efficiently.
3. Sort results by distance.
4. Use different data structures to explore performance and logic.

Data

Europe_cities.geojson

Exercise steps

1 – Load the geojson with the points. The points correspond to the center (centroid) of the following cities: Amsterdam, Berlin, Paris, Madrid, and Rome.

2 – Compute the distances among each pair of cities, in km, and store it into a list. You can select a subset of cities, and/or compute for it all, if you think it's feasible.

3 – Sort the list with the distances using two different algorithms. Measure the execution time (in milliseconds, not the Big O) of each and compare.

4 – Using two different implementations of searching algorithms, measure the computational time (in milliseconds, not the Big O) of each of them to find a selected city into the list.

5 – Now, let's make a test using a queue and a stack. Store the names of the cities into both structures, and compare the different ways of accessing it. You can use the following implementation to the data structures, or make your own (recommended for practicing). Try to make find a certain city on that, and measure the execution time. Compare both.

```
# Stack example
stack = []
stack.append("Amsterdam")
stack.append("Berlin")
print(stack.pop()) # LIFO

# Queue example
from collections import deque
queue = deque(["Paris", "Madrid"])
queue.append("Rome")
print(queue.popleft()) # FIFO
```

6 – Using the list of cities, and the distances computed, let's represent it into a connected graph where the edges contain the distances among pairs of cities. To represent the graph, you should select a subset of cities, and make a dictionary containing each connection, in the following format:

Graph = { city a: {city b: distance (a, b)}, city c: distance(a,c) ... }

For that, you should not create the dictionary writing yourself the names of the cities. Instead, make a function that reads the names of the cities and creates it.

7 – (extra, not mandatory) Plot that graph into a figure and see how it looks like.

8 – Finally, to understand how usefull KD-trees can be in geospatial algorithms, we will create a Kdtree for searching by the distance across **the whole set** of 720 cities. Select a few different cities, and compute the nearest neighbor. Check how many cities were checked until it found the nearest one. Measure also the computational time for that operation. For using the KD-tree, the package `scipy.spatial` provides a very efficient implementation: <https://docs.scipy.org/doc/scipy/reference/spatial.html>.

Deliverables

Submit a .zip with the code you implemented, containing the whole project folder, and a Readme.md file with your explanations and answers to the questions. Please, submit also the codes of the two exercises (equations) given in the lecture.